

Coursework: Integrating brain-inspired constraints in neural network models.

1a)

The first way to adjust the structural wiring of an artificial neural network is to vary the sparsity of the connections between neurons. Sparsity reduces how many connections each unit can have, so no neuron is densely connected to all other neurons. This matches the evidence that fewer than 1% of the potential synapse connections between neurons occur in cortical networks[1]. Neural networks can implement this by introducing a binary mask $\mathbf{M}$, where each entry $\mathbf{M}_{ij} \in \{0,1\}$ indicates whether a connection is possible between a presynaptic neuron j and a postsynaptic neuron i . The mask is applied over a recurrent weight matrix $\mathbf{W}_{dense}^{rec}$, whose entries, $(\mathbf{W}_{dense}^{rec})_{ij} \in \mathbb{R}$, represent the learned strength of influence of neuron j on neuron i and are updated using gradient descent. The sparse connectivity is given by:

$$\mathbf{W}_{sparse}^{rec} = \mathbf{M} \odot \mathbf{W}_{dense}^{rec} \quad [2]$$

where $\odot$ denotes the elementwise(Hadamard) product such that the sparse matrix $\mathbf{W}_{sparse}^{rec}$ has entries:
 $(\mathbf{W}_{sparse}^{rec})_{ij} = \mathbf{M}_{ij}(\mathbf{W}_{dense}^{rec})_{ij}$.

The number of ones in $\mathbf{M}$ controls the overall sparsity level. The lower the number of ones, the sparser the network is which imitates the brain-like wiring, whereas a higher number makes a denser network. As well as the fraction of ones and zeroes, the position also matters, since the brain contains about 60% more short-range connections than long-range connections[3]. To model this using $\mathbf{M}$, the ones should generally be close to each other with occasional ones further away to capture the sparsity and spatial structure of the connectivity within the brain. However, during the training process, the mask $\mathbf{M}$ can become too aggressive that it could lose accuracy, so the sparsity pattern must be chosen carefully to balance the modelling capacity and task performance.

The second way to alter the architecture to enforce brain-like constraints is to follow Dale's law. Dale's law states that a given neuron uses the same transmitter at all its synapses onto other neurons, so each neuron is either an excitatory or an inhibitory cell[4]. In a standard neural network, a single hidden unit can have a mix of positive and negative outgoing weights, implying it can excite some targets and inhibit others which doesn't comply with Dale's law. To give the architecture a brain-like constraint, the recurrent units can be divided into excitatory and inhibitory cells and the signs of all outgoing connections from each unit can be fixed depending on the cell type.

As with the sparsity, there is a recurrent weight matrix $\mathbf{W}^{rec}$, whose entries $\mathbf{W}_{ij}^{rec}$ define the influence of neuron j onto neuron i . To obtain a non-negative matrix $\mathbf{W}^{rec,+}$, the original learned weights matrix can be rectified using ReLU[4] as shown below:

$$\mathbf{W}^{rec,+} = \text{ReLU}(\mathbf{W}^{rec}), (\mathbf{W}^{rec,+})_{ij} = \begin{cases} \mathbf{W}_{ij}^{rec} & \text{if } \mathbf{W}_{ij}^{rec} \geq 0 \\ 0 & \text{if } \mathbf{W}_{ij}^{rec} < 0 \end{cases}.$$

A diagonal matrix $\mathbf{D}$ is then used to encode the cell type of each of the presynaptic neurons where $\mathbf{D}_{jj}$ is 1 if the unit is excitatory and -1 if the unit is inhibitory and then the Dale constrained weight matrix, $\mathbf{W}_{Dale}^{rec}$, is defined as the matrix product of $\mathbf{W}^{rec,+}$ and $\mathbf{D}$. If $\mathbf{D}_{jj}=1$, then every neuron in column j would be excitatory meaning it can only increase the activity of its targets, neurons i , and if $\mathbf{D}_{jj}=-1$, then every neuron in column j would be inhibitory, meaning that it would decrease the activity of neurons i .

The brain requires both excitatory and inhibitory cell types to work together so that neural activity does not cease nor overstimulates. Excitatory neurotransmitters increase the chance of neuron i firing an electrical impulse which is crucial for learning and memory, whereas inhibitory neurotransmitters reduce the likelihood of firing for controlling anxiety and calming down[5]. For the brain to function consistently, there must be a balance between the cell types which is controlled by $\mathbf{D}$, while the learned weights in $\mathbf{W}^{rec,+}$ determine the actual strength of excitation and inhibition each neuron receives. However, Dale's law reduces the flexibility by keeping neurons purely excitatory inhibitory which may lead to slower learning and worse performance[6].

b)

Coursework: Integrating brain-inspired constraints in neural network models.

The cost function is the error between the model's predictions to the real values. To impose brain-like, the cost function minimises the error across the training set by penalising solutions that do not follow the brain's structure. The neuro cost function J is expressed using the general formula:

$$J = J_{task} + \beta\varphi$$

where J_{task} is the standard cost function given by any sort of error calculation, such as RMSE, φ encodes a specific brain-like property, and β is the regularisation parameter that controls how strongly each constraint contributes to the total cost function.

One penalty that can influence the cost function to mimic brain-like constraints is sparsity. To bias a network towards the sparse connectivity in cortical circuits, an L1 penalty can be implemented on the weights, $|w_{j \rightarrow i}|$ [6]. This produces the sparsity-adjusted cost function [7]:

$$J_{sparsity} = J_{task} + \beta_{L1,weight} \sum_{j=1}^N \sum_{i=1}^N |w_{j \rightarrow i}|$$

$\beta_{L1,weight}$ controls how strongly sparse solutions influence the total cost function. Empirical studies have shown that very large $\beta_{L1,weight}$ values force extreme sparsity, increasing the cost function, causing too many connections to be removed. On the other hand, very small $\beta_{L1,weight}$ values produces dense networks which does not match the cortical connectivity [8]. This means $\beta_{L1,weight}$ must be tuned to balance task performance and its plausibility to match the brain-like sparsity constraint.

The second constraint revolves around the distance between neurons. In cortical networks, the probability of forming synapses decreases exponentially with the distance between neuronal units [9], since long-range connections have greater metabolic and wiring cost [10]. By assigning each unit a location in 3-dimensional space, the distance-dependent cost function [9] is given by:

$$J_{distance} = J_{task} + \beta_{WD} \sum_{j=1}^N \sum_{i=1}^N |w_{j \rightarrow i}| |d_{j \rightarrow i}|$$

where $|d_{j \rightarrow i}|$ is the distance between units i and j and β_{WD} is the L1 regularisation parameter for distance. This equation allows long-range connections to be penalised, reflecting the predominantly local connectivity present in cortical networks. If β_{WD} is too large, the model may underperform by suppressing too many long-range interactions, whereas a very small β_{WD} fails to signify any meaningful structure. This demonstrates the need to refine the value of β_{WD} to ensure a compromise [11] for both the accuracy and the realism of representing the structure of a brain.

The final constraint is the firing rate. In the brain, firing rate refers to the frequency at which neurons generate electrical impulses over time. Although cortical neurons can fire at rates from as little as 0.01 Hz to 100 Hz, [12] they generally spend long periods at lower activity levels to reduce the metabolic cost and to prevent excessive excitation. To follow this constraint in artificial neural networks, the cost function can include a term that penalises overly active units. To enforce this penalisation, an L2 penalty parameter is applied to the firing rate $a_i(t)$ of a neuron i at time t , as shown below:

$$J_{rates} = J_{task} + \beta_{L2,rates} \sum_{t=1}^N \sum_{i=1}^N a_i(t)^2 \quad [13].$$

The squared term penalises stronger activities more heavily, biasing the network towards lower average activity solutions. $\beta_{L2,ra}$ controls the regularisation of the firing rate. If $\beta_{L2,rates}$ gets too large, the firing rate decreases significantly [14], which could lead to underfitting and poor task performance despite following the constraints. Conversely, if $\beta_{L2,rates}$ is too small, then the firing rate may be too high, causing the model to fail to follow the brain's constraints, if the task loss. $\beta_{L2,rates}$ must be calibrated to achieve a trade-off between the accuracy and the ability to represent brain-like constraints.

c)

The learning rule controls how the weights of an artificial neural network are updated so that the model becomes more accurate over time [15], minimising errors between predicted and actual outputs. This is

Coursework: Integrating brain-inspired constraints in neural network models.

important for brain-like neural networks, since biological systems adapt continuously to incoming information. Learning rules in neural networks need to be able to replicate the learning rules used in brains.

There are multiple learning rules that can be used in neural networks. The first is backpropagation, a widely used algorithm. This involves propagating errors backwards through the network, using chain rule from calculus to calculate gradients and iteratively updating the weights and biases[16]. As useful as it can be in artificial neural networks, it falls short for modelling the brain's learning process. First, its weight updates in backpropagation depend on information from distant neurons which does not hold in the brain since a synapse only uses local variables such as membrane voltage, presynaptic and postsynaptic activity. Another reason is because backpropagation assumes the weights going forwards is equal to the weights going back, however this is not true for neurological systems since no same synapse is reused in reverse. The final reason is that backpropagation cannot do the acting (forward) and learning (propagation) phases at the same time, whereas the brain is able to do both simultaneously and continuously. These drawbacks make standard backpropagation not viable as a learning model for brain-like constraints[17].

A more suitable method is feedback alignment, an improved version of backpropagation that relaxes the characteristic that the forward weights are equal to the backward weights[18]. Feedback alignment propagates error signals backward through the network using fixed random feedback matrices rather than the forward weights as required by backpropagation. Despite this approximation, learning stays effective because the network gradually adapts the forward weights to make the random feedback useful[18]. However, feedback alignment still struggles to match the fact that it still uses non-local information, since synaptic updates require signals originating from downstream neurons rather than being computed purely by a presynaptic and postsynaptic neuron. As a result, even though feedback alignment improves its credibility to model the brain-like structure, the strict requirements for local learning and simultaneous acting and learning processes have not been met yet.

To address the limitations in both backpropagation and feedback alignment, the dendritic error model can be used as a learning method. A single neuron simultaneously propagates activity at the cell body, encodes error at the dendrites, and error propagation to the cell body, without the need for separate phases. The weight updates depend on the learning rate, the firing rate from the lower area, and the voltage difference between the dendrite and the cell body. This clears the major limitations of backpropagation and feedback alignment: it does both forward and backward processes simultaneously and the error is only obtained locally[17]. However, dendritic error models are still subject to a key setback: the architecture created by implementing the biological constraints may require high computational power. Despite that, the dendritic error model is currently more useful for cortical learning than backpropagation and feedback alignment.

The final learning rule is Random Feedback Local Online(RFLO), developed mainly for recurrent networks. RFLO approximates gradient-basing learning by starting from the backpropagation gradient, dropping a nonlocal term so that each synaptic update depends only on presynaptic and postsynaptic activity, where gradient descent is applied to this approximate gradient[18]. The weight change is given by the formula[17]:

$$\partial \mathbf{W}_{ab}(t) = \alpha [\mathbf{B}\delta(\mathbf{t})]_a p_{ab}(t)$$

where α is the learning rate, $\mathbf{B}$ is a random feedback matrix, $\delta(\mathbf{t})$ is the output error and $p_{ab}(t)$ is a moving average of recent co-activations between neurons a and b [16]. The main strengths are that RFLO only depends on local information, it does not need to reuse weights in the forward and backward directions, and it avoids rerunning the activations backwards at the end of the trial to learn[16]. However, RFLO is limited by the fact that co-activations are stored for a set period before they are removed at a rate:

$$\frac{dp_{ab}(t)}{dt} \propto -\frac{p_{ab}(t)}{\tau}$$

where τ is the time constant that sets how long a neuron's activity lasts for.

d)

Beyond architecture, cost function and learning rule, other aspects of how a network is trained can be altered to enforce brain-like constraints. The first characteristic is the training data and regime. Standard artificial neural networks are generally trained on randomly shuffled mini batches from a fixed dataset that the model can loop over. In contrast, biological neurons go through continual learning, a process where they retain past

Coursework: Integrating brain-inspired constraints in neural network models.

knowledge while receiving a constant stream of sensory inputs to adapt to new situations over time[19] as illustrated in Figure 1 below:



Figure 1: Comparison between standard training and continual learning. Standard uses the same data all the time, whereas continual learning uses new inputs for adaptation to new experiences.[24]

Training artificial neural networks so that they receive a continual supply of input information meets the brain-like constraint, so that the system must learn from ongoing experiences and cope with changes[19][21]. However, the main issue with this type of training is that it can cause catastrophic interference, where the training of the new input data interferes with the previously learned knowledge and causes a substantial drop in accuracy[22]. This necessitates the regularisation of important weights to stabilise old knowledge, which increases computational cost and reduces efficiency[22]. One way to parametrise this is by introducing a scalar variable ρ that sets the proportion of new to replayed data at each update. Smaller ρ values favour the use of old data, which is less brain-like but computationally cheaper[22], whereas larger ρ values denote a greater usage of the new data, which better matches the brain-like structure at the expense of greater computational demands and the increased risk of catastrophic interference. This highlights the need to balance these two conditions.

Another aspect is the temporal dynamics of neural activity and learning. In standard recurrent neural networks, the hidden state h_t is updated[18] as:

$$h_t = f(\mathbf{W}h_{t-1} + \mathbf{U}x_t)$$

where $\mathbf{W}$ is the recurrent weight matrix, $\mathbf{U}$ maps the input x_t to the hidden layer, and f is the nonlinear activation function. This equation implies that the hidden state has no explicit decay or time constant, meaning that the new hidden state is a direct nonlinear mapping from the previous hidden state and the current input. In real cortical networks, neurons behave as leaky integrators[18], accumulating synaptic input while the membrane potential slowly decays back towards a baseline over a time constant. To apply this constraint to the current hidden state equation, a decay term α can be added:

$$h_t = (1 - \alpha)h_{t-1} + \alpha f(\mathbf{W}h_{t-1} + \mathbf{U}x_t)$$

where $\alpha = \frac{\Delta t}{\tau}$ where τ is the time constant, as mentioned before[27]. For large values of τ , the value of α approaches zero, meaning that the decay rate slows down resulting in long-term memory, whereas for smaller values of τ , the decay rate increases, leading to short-term memory. This can be kept reasonable during training using the task loss:

$$J = J_{task} + \beta_{time} \sum_i (\tau_i - \tau_{true})^2$$

where β_{time} penalises the deviations in the time constant. Extremely small or large deviations will be penalised strongly, hence β_{time} is able to control the balance between the task performance and the realistic temporal dynamics.

2a)

As a base recurrent neural network(RNN), Gated Recurrent Unit(GRU) is used for its effectiveness in capturing long-term dependencies in sequential data[28], a limitation that traditional RNNs have. Traditional RNNs struggle with learning and maintaining long-term dependencies due to the vanishing gradient problem, which

Coursework: Integrating brain-inspired constraints in neural network models.

arises from gradients diminishing as they propagate through layers or time steps. This resulted in RNNs' limited ability of retaining information over long periods of time, making them unreliable for longer time sequences.

To address the issue, Long Short-Term Memory(LSTM) networks and GRUs were developed. They both specialise in using gated mechanisms to regulate the flow of information over time, enabling them to retain information for greater durations.

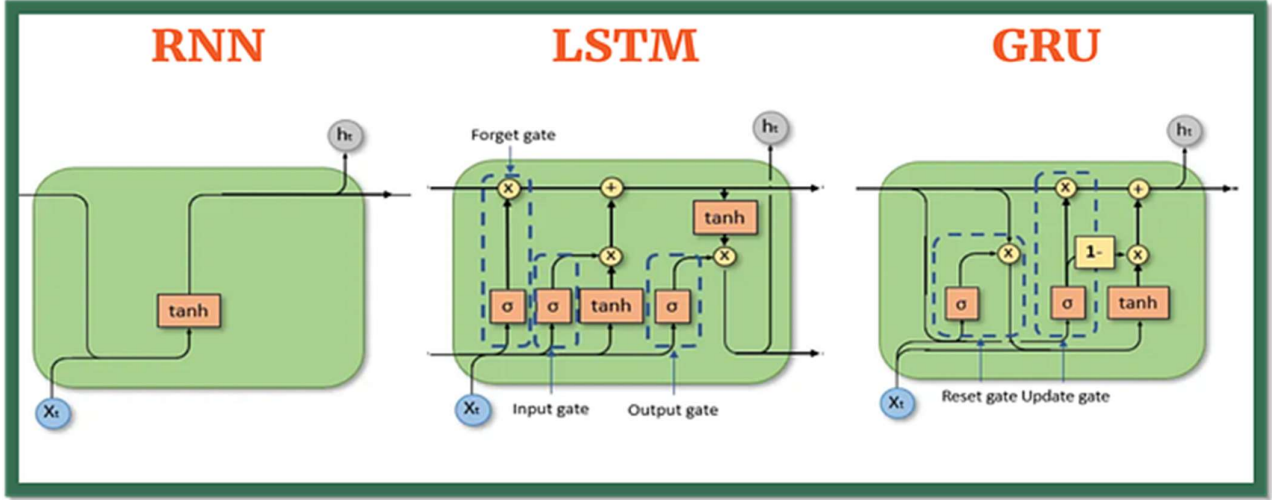


Figure 2: Comparison of the schematic structures of traditional RNNs, LSTMs and GRUs. Each diagram shows the activation functions used, the hidden states and the inputs. LSTM and GRU both show the gates present in their structures.[20]

Figure 2 illustrates that LSTMs use three gates to control the information: the input gate, which determines how much new information should be stored in the memory cell; the forget gate, which controls how much information is discarded from memory cell; and the output gate, that decides the next hidden state[28]. While this mechanism is more effective than traditional RNNs in learning and retaining long-term dependencies, the computational requirements are significantly higher, making them less efficient. As a result, this may lead to LSTMs being impractical for certain applications, especially where computational resources are limited.

In contrast, GRUs offer a more suitable alternative. They simplify the LSTM structure by using only two gates. The update gate is used to decide how much of the previous information must be kept, and the reset gate chooses how much information should be forgotten at each time step. This makes GRUs a more suitable choice, given that the computational resources are restricted.

As previously discussed, the update gate z_t controls the amount of information from the previous hidden state that is retained for the next time step. The formula for the update gate is as follows:

$$z_t = \sigma(\mathbf{W}_z[h_{t-1}, x_t]) \quad [20]$$

where σ is the sigmoid activation function that governs the proportion of the previous hidden state h_{t-1} is to be retained for the next hidden state h_t , $\mathbf{W}_z$ is the weight matrix for the update gate which is applied to the concatenation of h_{t-1} and the current input x_t . The reset gate r_t determines how much of the information is dispelled, calculated by:

$$r_t = \sigma(\mathbf{W}_r[h_{t-1}, x_t]) \quad [20]$$

where $\mathbf{W}_r$ is the weight matrix for the reset gate. The other characteristics of GRUs include the candidate hidden state $\tilde{h}_t$, which is the predicted new hidden state calculated using the previous weight matrix applied to the reset gate $\mathbf{W}_h$, h_{t-1} and x_t as given by the formula:

$$\tilde{h}_t = \tanh(\mathbf{W}_h \cdot [r_t \cdot h_{t-1}, x_t]) \quad [20]$$

The final characteristic of GRUs is the final hidden state, which is calculated by:

$$h_t = (1 - z_t) \cdot h_{t-1} + z_t \cdot \tilde{h}_t \quad [20]$$

Overall, the gates work together to compute the final hidden state, providing a computationally effective way to capture extended temporal relationships.

Coursework: Integrating brain-inspired constraints in neural network models.

Using the adjustment in architecture to account for sparsity, a sparse GPU allows for a more accurate model of the cortical structure, since the brain exhibits sparse connectivity patterns and energy-efficient network. This model retains the core functionality of the GRU, keeping the update and reset gate, but it incorporates a sparsity mask matrix $\mathbf{M}$, which contains binary inputs, denoting whether a connection has a probability of occurring. The binary values depend on the sparsity constant ζ , which represents the probability that a weight connection will remain active. The dense weight matrix $\mathbf{W}$ contains the entries $\mathbf{W}_{ij}$, which are random numbers generated between 0 and 1. If the random number is less than ζ , the corresponding entry in the mask matrix is set to 1, implying the neuron is active, otherwise, it is set to 0. This is also written as:

$$\mathbf{M}_{ij} \sim \text{Bernoulli}(\zeta)$$

The sparsity mask matrix is formed and is elementwise multiplied by the dense weight matrix $\mathbf{M}_{ij}$ to get $\mathbf{W}_{sparse}$ as computed previously. This sparsity constraint maintains the biological realism that the original GRU does not hold, by limiting the number of active connections at a given time step.

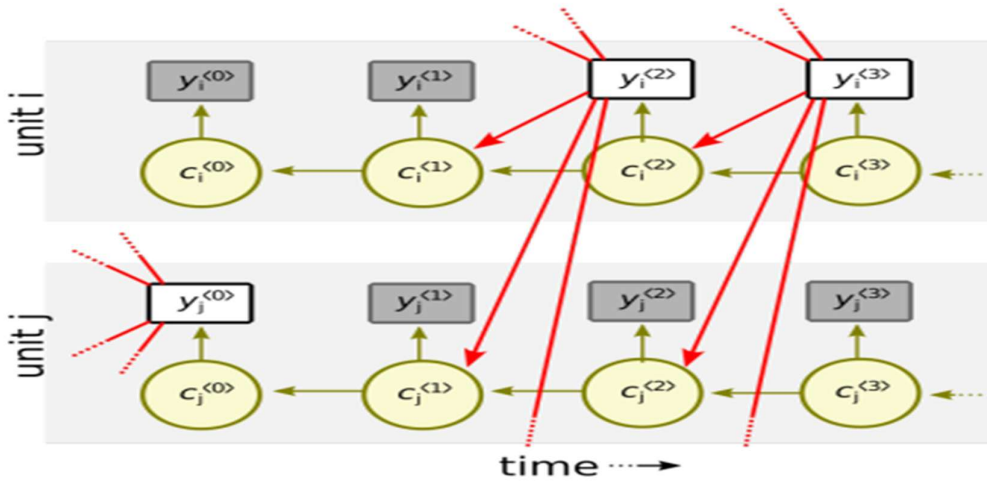


Figure 3: Sparse dynamics for two GRU units i and j [30]

Figure 3 demonstrates the general structure of a GRU with sparse connections between units. y_i and y_j denote the output, and c_i and c_j represents the internal state of the unit at different time steps. The red arrows show the presence of connections between the two units across different time steps. There are minimal connections between units i and j which could influence the learning process. As the number of connections reduce, the model may focus on learning the most relevant relationships between neurons which allows the network to be more resistant to noise effects and potentially allow it to generalise better to unseen data. The reduction in connections also reduces the likelihood of overfitting since it reduces the model's capacity to memorise events instead of learning new events.

b)

The standard GRU and the sparse GRU were compared to assess the impact of introducing a brain-like constraint on model performance and efficiency. The core difference between the two models is the number of active connections in each model. This constraint aims to force the model to rely on a less dense representation of the cortical structure, potentially improving generalisation.

To measure the performance of the models, the cross-entropy loss was used as an evaluation metric. The cross-entropy loss measures the difference between the predicted probabilities and the true labels. Specifically, it is calculated as:

$$\text{loss}(x, y) = - \sum y \log(x)$$

Coursework: Integrating brain-inspired constraints in neural network models.

where x is the predicted probability for a given class and y is the true label for a class[31]. The formula penalises the model when it predicts high probabilities for incorrect classes. In the code, the sparsity constant

```
class GRUNet(nn.Module):
    """Standard GRU Network."""
    def __init__(self, input_size, hidden_size, output_size, num_layers=1):
        super(GRUNet, self).__init__()
        self.gru = nn.GRU(input_size=input_size, hidden_size=hidden_size, num_layers=num_layers, batch_first=False)
        self.fc = nn.Linear(hidden_size, output_size)

        self.activations = [] # To store activations

    def forward(self, x):
        gru_output, hn = self.gru(x) # GRU returns only the hidden states
        self.activations.append(gru_output.detach().cpu().numpy()) # Store activations (detached from the graph)
        out = self.fc(gru_output)
        return out, gru_output

class SparseGRU(nn.Module):
    def __init__(self, input_size, hidden_size, output_size, sparsity_level=0.01, num_layers=1, sparsity_enabled=True):
        super(SparseGRU, self).__init__()
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.sparsity_level = sparsity_level
        self.num_layers = num_layers
        self.sparsity_enabled = sparsity_enabled

        self.gru = nn.GRU(input_size=input_size, hidden_size=hidden_size, num_layers=num_layers, batch_first=False)
        self.fc = nn.Linear(hidden_size, output_size)

        self.activations = [] # To store activations

    def apply_sparse_mask(self):
        weight_ih, weight_hh = self.gru.weight_ih_l0, self.gru.weight_hh_l0
        mask_ih = torch.rand_like(weight_ih) < self.sparsity_level
        mask_hh = torch.rand_like(weight_hh) < self.sparsity_level
        self.sparsity_mask = (mask_ih, mask_hh)
        self.gru.weight_ih_l0.data *= mask_ih.float()
        self.gru.weight_hh_l0.data *= mask_hh.float()

    def forward(self, x):
        self.apply_sparse_mask()
        gru_output, hn = self.gru(x)
        self.activations.append(gru_output.detach().cpu().numpy()) # Store activations (detached from the graph)
        out = self.fc(gru_output)
        return out, gru_output
```

Figure 4: Code snippet to show the classifications for standard GRU and Sparse GRU

was introduced to control the number of zeroed weights in the sparse GRU model. The implementation is shown below in the code snippet.

The sparsity was able to be controlled by a parameter to experiment its effect on the model. The task used was Pulse Decision Making - a pulse-based decision-making task where discrete stimuli are presented briefly as pulses - from the NeuroGym environments selection to gain a clear understanding of how the sparsity affected

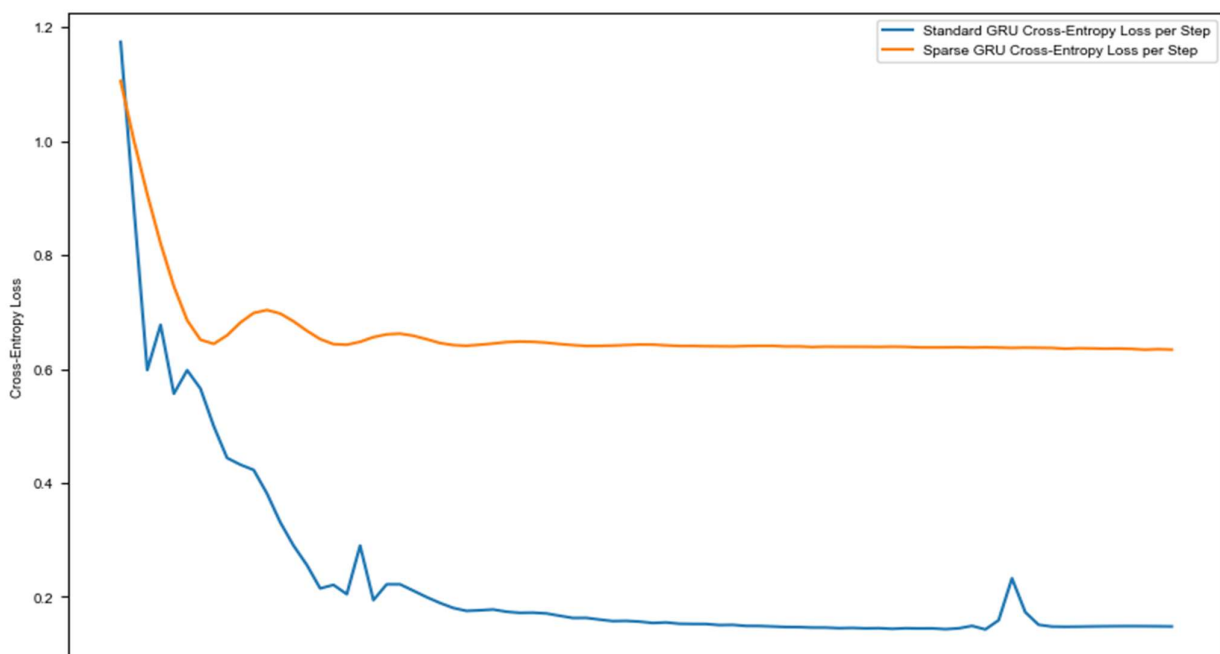


Figure 5: Progression of cross-entropy loss over time steps for Standard and Sparse GRU models

Coursework: Integrating brain-inspired constraints in neural network models.

total performance. A ζ of 0.01 in this code meant that the model was 1% dense, which modelled the brain a lot better since the cortical neurons in the brain form a very sparse network.

Figure 5 shows that the sparse and the standard GRU models started at similar points however, during the entire period, the standard GRU model's accuracy decreases significantly to approximately 0.2, whereas the sparse GRU model stays at around 0.63. This is what we expected since the sparser a network is, the fewer parameters there are, hence there is more room for error, which reduces chances of overfitting, making the learning process more meaningful than with the dense GRU model which has a low error since there are so much more parameters to help memorise the training data, which is a strong case of overfitting.

To further understand the effect of the sparsity model, various values of the ζ was used to see how they would vary the cross-entropy error which is shown in Figure 6. The ζ was given various values : $\zeta = [0, 0.3, 0.6, 0.9]$ for comparison.

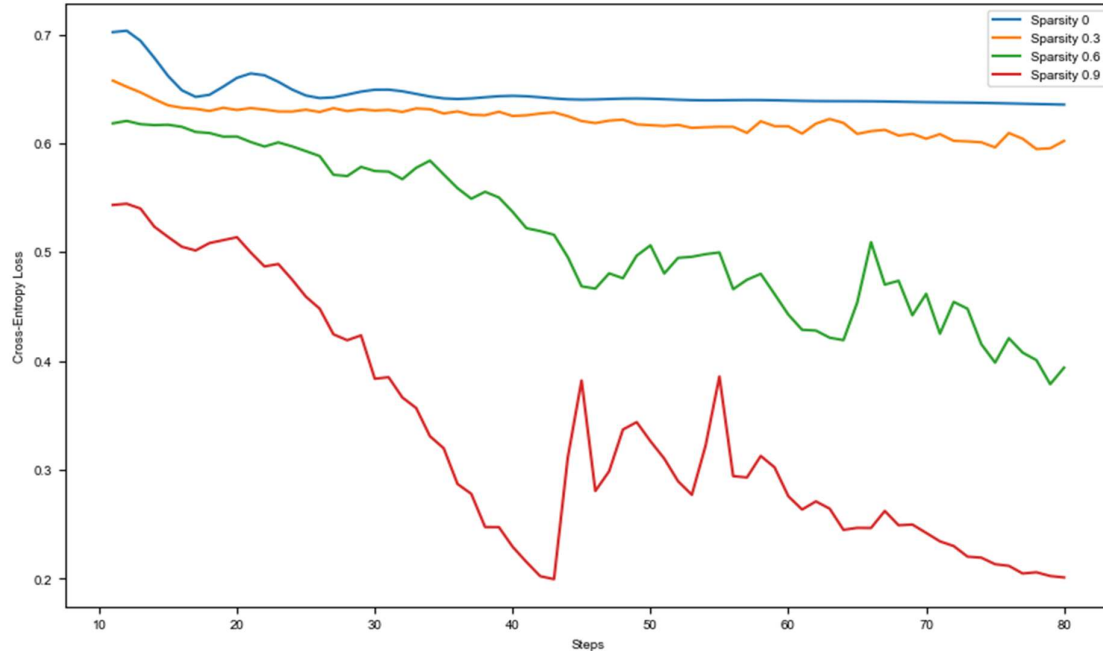


Figure 6: Effects of varying the sparsity constant on the performance of a sparse GRU

The graph shows that the results align with the expectations that the sparser a network, the fewer parameters it has, creating more room for error, which also reduces chances of overfitting. The model is forced to focus on more important features in the data, leading to a more meaningful process.

To allow for further comparison, the computational efficiency of the two models to see if the additional constraint had much of an effect. Table 1 shows the computational difference between the two models. For 500 steps, GRU was able to complete the learning process in 25.2 seconds which is a notable bit higher than the sparse GRU, which completed its process in 21.1 seconds. The difference of 4.1 seconds in training time might seem small, but it is notable when considering that the training was conducted on a dataset with 100 timesteps and 16 batch samples. For a much larger dataset, this difference becomes significant.

Model	Time taken (s)
Standard GRU	25.2
Sparse GRU	21.1

Table 1: Time taken for standard and sparse GRU models to run 500 steps

Coursework: Integrating brain-inspired constraints in neural network models.

The reduction of the total number of nonzero parameters led to fewer computations per step, which resulted in faster processing times. This was also seen when comparing the training times over the same ζ values as compared in Figure 6. The table below shows the outcome of the times taken for each model to finish training:

ζ value	Time taken to run(s)
0	27.5
0.3	24.2
0.6	22.4
0.9	21.3

Table 2: Comparison of how change in ζ varies the time taken to run a training process.

The table again shows how as ζ increases, the processing time decreases.

The comparison between GRU and Sparse GRU models showed that the brain-imposed constraint for sparsity is effective in enhancing model performance and efficiency. Despite increasing the error function, it was balanced by the realism that it showed to model an actual cortical system, as well as increasing efficiency. By reducing the number of parameters, the model is forced to rely on less dense representations allowing it to be reliably used in modelling the brain's system. During the modelling and training process, several key hyperparameters were fixed to ensure a fair comparison, each of which plays a crucial role in how the models are trained and how they perform.

The learning rate was kept the same for both models to ensure that differences in their learning processes were not due to how quickly each model adapted. Similarly, both models had the same hidden size and number of layers, ensuring that the models had equivalent capacity for learning and representing data. By controlling these variables, we were able to isolate the effect of sparsity, specifically, how the reduction in active parameters influenced generalisation, overfitting, and training efficiency.

Overall, the results highlight that sparsity can significantly improve the performance and computational efficiency of neural networks, making them better suited for tasks where generalisation and efficiency are crucial.

2c)

The second task chosen from the NeuroGym environment is the Delay Match Sample task. During the sample period, a stimulus is presented, characterised by a one-dimensional variable, such as its orientation, ranging from 0 to 360 degrees. After a delay period, a test stimulus is shown. The agent must then determine if the sample and test stimuli are identical and report its decision during the decision period.

When applying the same method to the second task, it did still generally follow the idea that the sparse model had greater error. The sparse GRU drops sharply from 1.05 to 0.8, then gradually decreases to about 0.76, not exhibiting much movement. However, when looking at the standard GRU model, it also displays the same massive drop to 0.2, however it shows more erratic movement. This can occur due to multiple reasons, such as excessive learning rate, the exploration of the parameter space could have led to exploding gradients or the task complexity. . Despite the brief fluctuations, the overall trend supports the observation that the Sparse GRU model tends to have higher error than the Standard GRU model, which aligns with the expectation that the denser model (Standard GRU) has more capacity to learn and retain information, especially for more complex tasks like Delay Match Sample.

Coursework: Integrating brain-inspired constraints in neural network models.

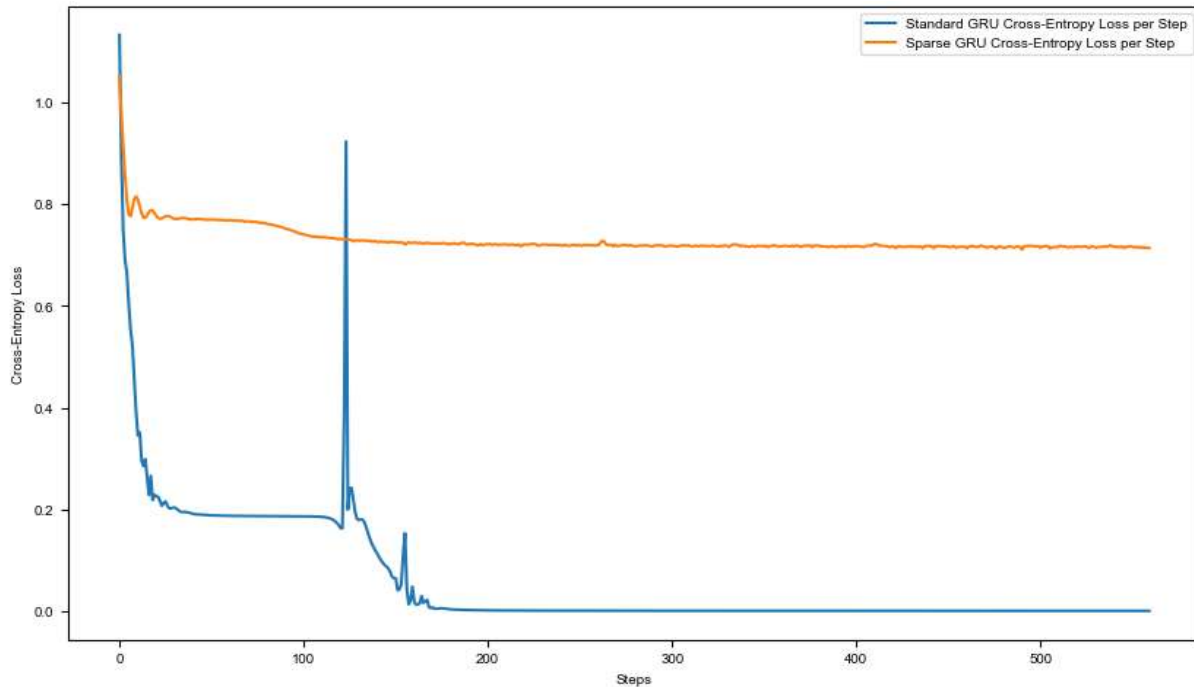


Figure 7: Cross-entropy loss for standard and sparse GRU models, completing the Delay Match Sample task

Furthermore, the times taken to complete the tasks are shown below:

Model	Time to run(s)
Standard	30.5
Sparse	28.6

Table 2: Comparison between Standard and Sparse GRU models to run the Delay Match Sample task

These times are shown to be slightly longer than those of the pulse-based decision-making task, which is to be expected since this task involves a memory retention component, where the agent needs to store information about the sample stimulus during the delay period and compare it with the test stimulus. The dense model contains more parameters, meaning that more weights need to be updated during each training step, causing the overall computation time to increase. Meanwhile, the sparse model has fewer active parameters, resulting in less weight training, lowering the computational cost. Comparing these times to the previous task's times, at greater scales regarding the amount of data present, the computation cost will be significant.

Looking at the different sparsity levels, the trend still follows for the Delay Match Sample task however, they clearly fluctuate a lot more than the previous task. The fluctuations are a result of poor weight initialisation or noise which could suggest that this task may not be ideal for the model, which may suggest that the memory requirements of this task are challenging for the sparse architecture, especially when it comes to maintaining consistency in decision-making over time.

While the sparse model's training time is reduced, the sparsity constant plays a crucial role in determining the model's performance and efficiency. As with the previous task, the sparsity constant had a direct effect on the cross-entropy loss and computational efficiency.

For the Delay Match Sample task, the influence of varying sparsity constants is visible as shown in Figure 8, and the loss curves displayed more erratic fluctuations than in the Pulse Decision Making task. As expected, increasing sparsity—therefore reducing the number of active weights—led to an increase in the cross-entropy

Coursework: Integrating brain-inspired constraints in neural network models.

loss. However, the loss curves for the Delay Match Sample task exhibited a greater degree of fluctuation compared to the previous task, indicating that the sparse architecture may struggle more with this task's complexity. When the $\zeta = 1$, all the weights in the network are zeroed out. This means the network is unable to learn anything because there are no active weights to adjust during backpropagation. No learning or adaptation can occur because the gradients that are supposed to update the weights are now irrelevant.

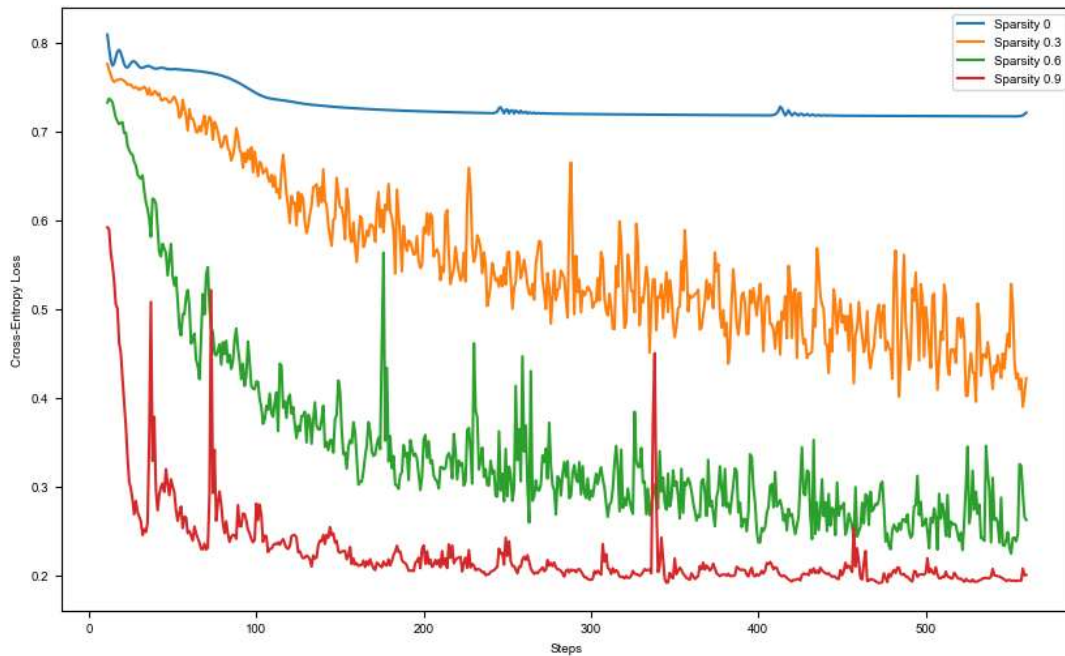


Figure 8: Effects of varying the sparsity constant on the performance of a sparse GRU for completing Delay Match Sample task

The results from experimenting with different sparsity constants highlight the trade-off between model performance and computational efficiency. While the sparse model offers reduced computational time, it comes at the cost of performance, particularly in more complex tasks that require memory retention and temporal reasoning. Dense models tend to outperform the sparse ones in such cases, as they have more capacity to handle the complexity of the task. For tasks that are less dependent on memory, sparse models may still be effective, but for tasks like Delay Match Sample, the model's limited parameter set appears to struggle.

In addition to analysing the cross-entropy loss, the hidden unit activity over time was analysed to understand how the models process and retain information during the task. Specifically, we can track the activations of individual hidden units during the Delay Match Sample task to see how well the models handle the memory retention aspect of the task. Figure 9 below shows the variation in the activation of a hidden unit over time for both models. The activations exhibit fluctuations within a certain range, reflecting how the hidden state evolves as the model processes the task. These fluctuations can be attributed to how each model retains and updates information over the course of the task, particularly during the delay period where memory retention is critical. The standard GRU model's hidden unit peaks at 0.16, which is significantly higher than that of the sparse GRU model. This suggests that the standard GRU model can store and retrieve information from the hidden state with greater magnitude, likely benefiting from its larger number of parameters. This could lead to higher capacity for memorizing and recalling features of the stimuli, which is particularly useful in a task like Delay Match Sample where maintaining memory over time is crucial.

Coursework: Integrating brain-inspired constraints in neural network models.

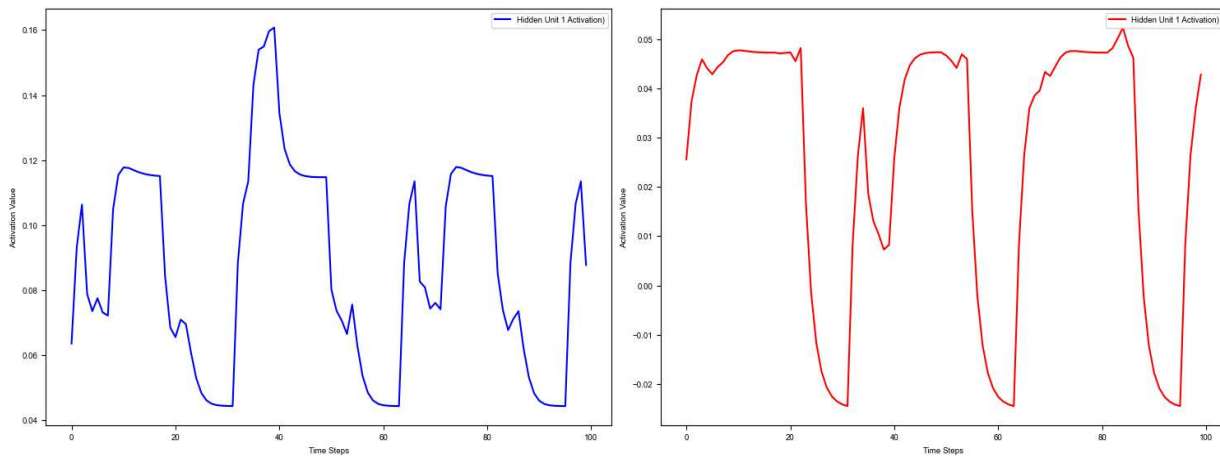


Figure 9: Comparative plot of the hidden activation of a single hidden unit over time. Left – Standard GRU; right – Sparse GRU

The standard GRU model has a higher standard deviation of 0.031, while the sparse GRU model has a slightly lower standard deviation of 0.027. This indicates that the standard GRU shows slightly greater variability in its activations, reflecting more pronounced fluctuations in the hidden state over time. In contrast, the sparse GRU model's smaller standard deviation suggests that its activations are more stable, likely due to the fewer active parameters, which could lead to more consistency in the hidden unit's behaviour. This indicates that while both models retain information, the standard GRU model has greater capacity and variability in its representation, whereas the sparse GRU model demonstrates more consistent but potentially less flexible memory processing.

d)

By using two tasks instead of one, the total loss curve shown in Figure 10 indicates how the model's performance improves over time as it learns from both tasks. Initially, the loss is quite high, at around 2.2, but it steadily decreases with each training step. By the end of the training process, the total loss stabilises at around 1.2. This suggests that the model has made significant progress in learning both tasks, with the combined loss of the tasks steadily decreasing as the model adjusts its parameters. The smooth, steady decline indicates that the model is adapting well to the multi-task learning environment, with both tasks contributing to a reduction in the loss. However, it's important to note that the overall total loss is still relatively high, which may indicate that the model is struggling with fully optimising performance for both tasks simultaneously.

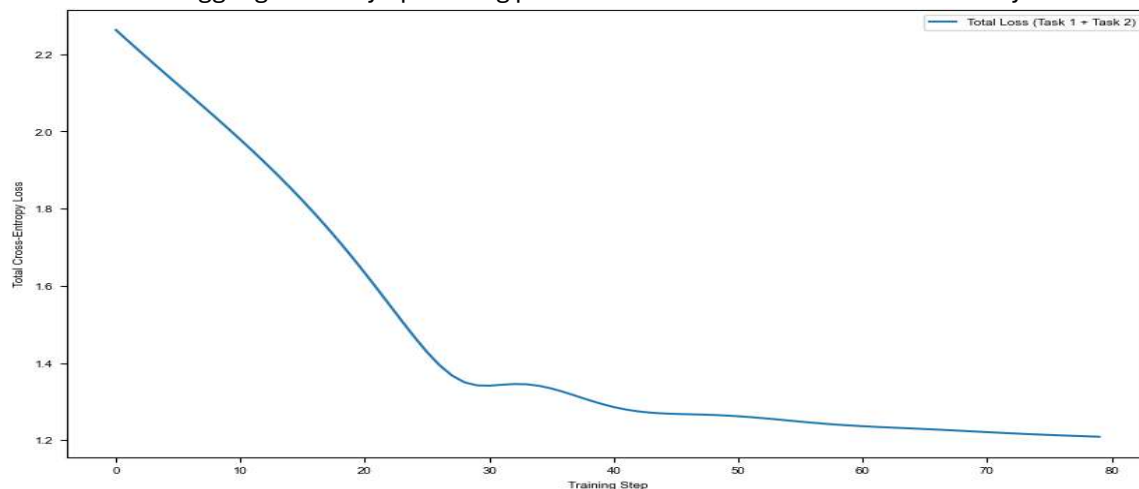


Figure 10: Total cross-entropy loss for the multi-task learning model over 80 training steps, showing the combined loss from Task 1 (Delay Match Sample) and Task 2 (Perceptual Decision Making).

Coursework: Integrating brain-inspired constraints in neural network models.

Figure 11 shows the activation of Hidden Unit 1 over time during the multi-task training process.

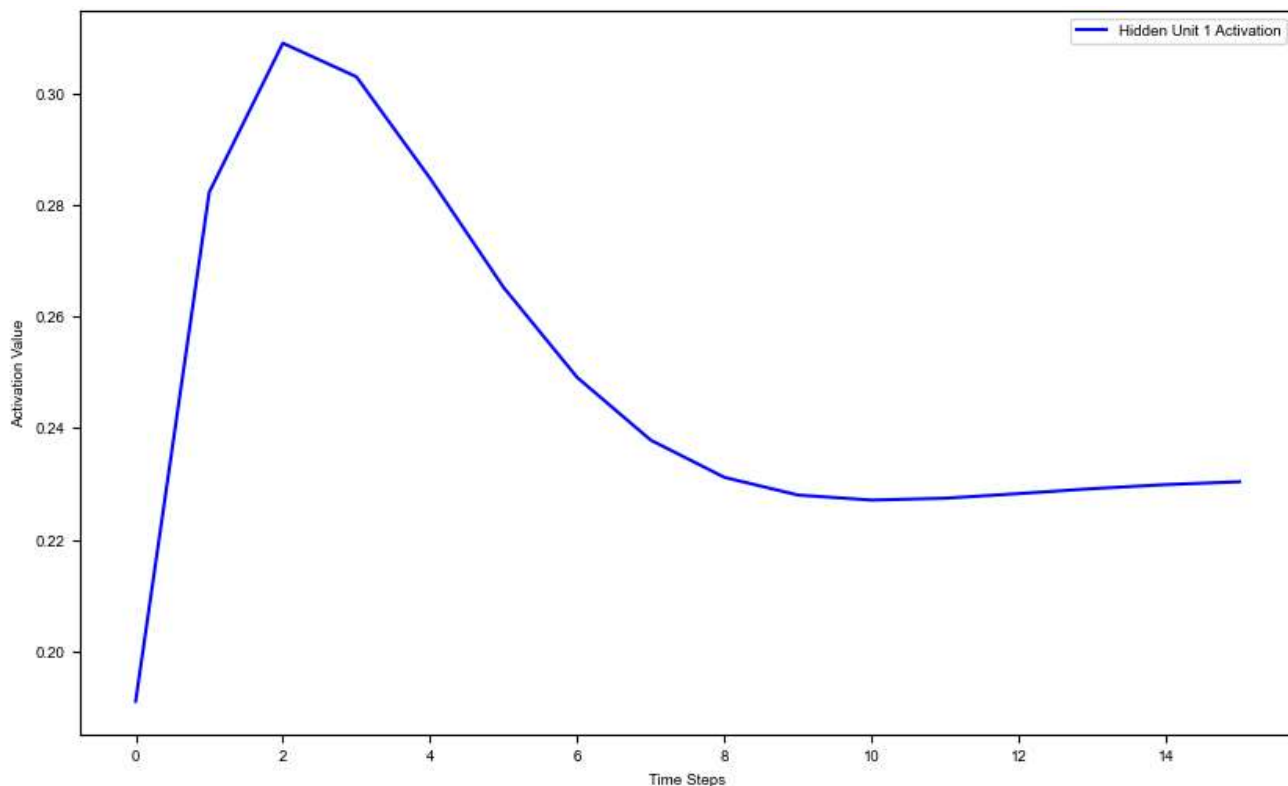


Figure 11: Activation values of Hidden Unit 1 over time during multi-task training.

This activation curve reveals several key stages in the model's learning behaviour. At the start of the training, the activation value spikes to around 0.33. This sharp increase suggests that the model undergoes a significant shift in its internal state as it begins processing the task-specific information. Such a spike is typical in the early stages of training, where the model is exploring and adjusting its weights to better respond to the new inputs and optimize its internal representations. Following the initial spike, the activation begins to decrease and stabilizes between 0.20 and 0.25. This gradual decline indicates that the model is refining its learning process, moving from an exploration phase to a more stable state. The reduction in activation represents the model's adjustment, where it adjusts its internal states to better represent the task and optimise memory retention and decision-making over time. Once the activation reaches a more stable range, it remains relatively constant. This suggests that the hidden unit is now steadying, and the model has learned the required temporal patterns needed for the tasks. The consistency in activation implies that the model's learning process has reached a certain level of efficiency, yet it may also indicate that the model has stopped adapting further, possibly because it has hit a performance plateau or has converged to an optimal solution for the given tasks.

This pattern in the hidden unit's activation reflects the typical training dynamics of deep learning models, where initial exploration is followed by refinement, and eventually, a maintenance phase. However, the plateau-like behaviour might also suggest room for further optimisation or adjustments to enhance the model's learning capacity for both tasks.

Coursework: Integrating brain-inspired constraints in neural network models.

3)

Through this analysis of multi-task learning with brain-inspired neural network architectures, we explored how different modifications to the standard neural network model can be made to align with brain-like constraints, and how these changes impact the performance and training of neural networks on tasks such as memory matching and perceptual decision-making.

In the first part of the coursework, we discussed several ways to impose brain-like constraints on neural networks. These included adjusting the architecture by introducing sparsity and dividing neurons into excitatory and inhibitory categories, inspired by the structural properties of the brain's neurons and their connectivity. Sparsity, specifically, mimics the limited connectivity observed in biological networks, where less than 1% of potential synapses are formed between neurons. Our implementation of sparsity in the network reduced computational costs but came at the expense of model performance, especially on more complex tasks requiring memory retention, as seen in the Delay Match Sample task. Moreover, adhering to Dale's law, which posits that neurons are either excitatory or inhibitory, ensured that the network learned with a structure closer to the brain's dynamics, where neural balance is critical for function.

The cost function design also played a crucial role in enforcing these constraints. We incorporated L1 and L2 regularization to promote sparsity and penalize overly active neurons, respectively. The adjustments ensured that the model adhered to the brain's natural tendency to minimize metabolic costs by reducing unnecessary activity. While the regularization improved the alignment of the network with biological principles, it also added complexity in tuning the model's performance.

The learning rule, specifically the feedback alignment model, was explored as an alternative to traditional backpropagation. This change allowed the model to learn using local information rather than relying on global error signals, more closely mimicking the brain's approach to synaptic learning. Despite this improvement, it still did not fully capture the real-time, continuous learning abilities of biological systems. The dendritic error model, an advancement in learning rules, is a potential future direction to better approximate the brain's ability to learn in real-time, simultaneously updating weights and processing information.

In the second part of the coursework, we trained and compared multiple models using two tasks from the NeuroGym environment: Delay Match Sample and Perceptual Decision Making. The multi-task learning approach, which utilized shared layers across both tasks, demonstrated that the brain-inspired models learned effectively, although their performance was slightly lower compared to more traditional, fully-connected networks. This suggests that while brain-inspired constraints (such as sparsity and excitatory-inhibitory balance) help in aligning neural networks with biological systems, they also impose trade-offs in computational efficiency and task performance, particularly for more complex tasks.

In conclusion, this coursework demonstrated the potential of integrating brain-inspired modifications into neural networks. While such modifications align models more closely with biological principles, they come with inherent trade-offs that must be carefully managed. Further research into more sophisticated learning rules and architectures could help strike a better balance between computational efficiency and biological plausibility, advancing the field of neuro-inspired machine learning.

Coursework: Integrating brain-inspired constraints in neural network models.

References

- [1] - Fruengel, R. and Oberlaender, M. (2025). Sparse connectivity enables efficient information processing in cortex-like artificial neural networks. *Frontiers in Neural Circuits*, 19.
doi:<https://doi.org/10.3389/fncir.2025.1528309>.
- [2] - Jaiswal, A., Ma, H., Chen, T., Ding, Y. and Wang, Z. (n.d.). *Training Your Sparse Neural Network Better with Any Mask*. [online] Available at: <https://proceedings.mlr.press/v162/jaiswal22a/jaiswal22a.pdf> [Accessed 1 Dec. 2025].
- [3] - <https://aiimpacts.org/author/tegan-mccaslin> (2018). *Transmitting fibers in the brain: Total length and distribution of lengths*. [online] AI Impacts. Available at: <https://aiimpacts.org/transmitting-fibers-in-the-brain-total-length-and-distribution-of-lengths/#easy-footnote-bottom-9-1118>.

Coursework: Integrating brain-inspired constraints in neural network models.

- [4] - Cornford, J., Kalajdzievski, D., Leite, M., Lamarquette, A., Kullmann, D. and Richards, B. (n.d.). *Published as a conference paper at ICLR 2021 LEARNING TO LIVE WITH DALE'S PRINCIPLE: ANNS WITH SEPARATE EXCITATORY AND INHIBITORY UNITS*. [online] Available at: https://discovery.ucl.ac.uk/id/eprint/10167375/1/3565_learning_to_live_with_dale_s_p.pdf [Accessed 3 Dec. 2025].
- [5] - Guy-Evans, O. (2025). *Excitatory Vs. Inhibitory Neurotransmitters*. [online] Simply Psychology. Available at: <https://www.simplypsychology.org/excitatory-vs-inhibitory-neurotransmitters.html>.
- [6] - Barranca, V.J., Bhuiyan, A., Sundgren, M. and Xing, F. (2022). Functional Implications of Dale's Law in Balanced Neuronal Network Dynamics and Decision Making. *Frontiers in Neuroscience*, [online] 16, p.801847. doi:<https://doi.org/10.3389/fnins.2022.801847>.
- [7] - Yang, G.R., Joglekar, M.R., Song, H.F., Newsome, W.T. and Wang, X.-J. (2019). Task representations in neural networks trained to perform many cognitive tasks. *Nature Neuroscience*, [online] 22(2), pp.297–306. doi:<https://doi.org/10.1038/s41593-018-0310-2>.
- [8] - Wen, M., Xu, Y., Zheng, Y., Yang, Z. and Wang, X. (2021). Sparse Deep Neural Networks Using $L1, \infty$ -Weight Normalization. *Statistica Sinica*. doi:<https://doi.org/10.5705/ss.202018.0468>.
- [9] - Achterberg, J., Akarca, D., Strouse, D.J., Duncan, J. and Astle, D.E. (2023). Spatially embedded recurrent neural networks reveal widespread links between structural and functional neuroscience findings. *Nature Machine Intelligence*, [online] pp.1–13. doi:<https://doi.org/10.1038/s42256-023-00748-9>.
- [10] - Jamadar, S.D., Behler, A., Deery, H. and Breakspear, M. (2025). The metabolic costs of cognition. *Trends in Cognitive Sciences*. [online] doi:<https://doi.org/10.1016/j.tics.2024.11.010>.
- [11] - Achterberg, J., Akarca, D., Strouse, D.J., Duncan, J. and Astle, D.E. (2023). Spatially embedded recurrent neural networks reveal widespread links between structural and functional neuroscience findings. *Nature Machine Intelligence*, [online] pp.1–13. doi:<https://doi.org/10.1038/s42256-023-00748-9>.
- [12] - Roxin, A., Brunel, N., Hansel, D., Mongillo, G. and van Vreeswijk, C. (2011). On the Distribution of Firing Rates in Networks of Cortical Neurons. *Journal of Neuroscience*, 31(45), pp.16217–16226. doi:<https://doi.org/10.1523/jneurosci.1677-11.2011>.
- [13] - Vishwa Goudar, Peysakhovich, B., Freedman, D.J., Buffalo, E.A. and Wang, X.-J. (2023). Schema formation in a neural population subspace underlies learning-to-learn in flexible sensorimotor problem-solving. *Nature Neuroscience*, 26(5), pp.879–890. doi:<https://doi.org/10.1038/s41593-023-01293-9>.
- [14] - Trotta, F. (2022). *Understanding l1 and l2 Regularization | Towards Data Science*. [online] Towards Data Science. Available at: <https://towardsdatascience.com/understanding-l1-and-l2-regularization-93918a5ac8d0/>.
- [15] - S.K. Jeswal, Snehashish Chakraverty (2021), Chapter 10 - Fuzzy eigenvalue problems of structural dynamics using ANN, <https://www.sciencedirect.com/science/chapter/edited-volume/pii/B9780128221334000104>
- [16] - GeeksforGeeks (2024). *Backpropagation in Neural Network*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/machine-learning/backpropagation-in-neural-network/>.
- [17] - Song, Y., Lukasiewicz, T., Xu, Z. and Bogacz, R. (2020). Can the Brain Do Backpropagation? —Exact Implementation of Backpropagation in Predictive Coding Networks. *Advances in neural information processing systems*, [online] 33, p.22566. Available at: <https://pmc.ncbi.nlm.nih.gov/articles/PMC7610561/>.
- [18] - Arild Nøkland, 2016, Direct feedback alignment provides learning in deep neural networks, <https://dl.acm.org/doi/10.5555/3157096.3157213#:~:text=A%20recently%20discovered%20method%20called,the%20permutation%20invariant%20MNIST%20task.>

Coursework: Integrating brain-inspired constraints in neural network models.

[19] - German I. Parisi¹ , Ronald Kemker² , Jose L. Part³ , Christopher Kanan² , Stefan Wermter¹, 2019, Continual Lifelong Learning with Neural Networks: A Review, <https://arxiv.org/pdf/1802.07569>

[20] - Anishnama (2023). *Understanding Gated Recurrent Unit (GRU) in Deep Learning*. [online] Medium. Available at: <https://medium.com/@anishnama20/understanding-gated-recurrent-unit-gru-in-deep-learning-2e54923f3e2>.